MBA Tech Artificial Intelligence

OOPS mini project

MUMBAI BUS TRANSIT MANAGEMENT SYSTEM

An Object-Oriented Contactless Smart Card, 2D QR Code Automated Ticketing, Dynamic Fare Calculation & Audit Intelligence System

R013. Sai Chaudri	70512500010
R020. Tisya Gawande	70512500077
R022. Daivik Guha	70512500001

ABSTRACT

Urban transit systems in high-density metropolitan areas like Mumbai represent critical civic lifelines. The Brihanmumbai Electric Supply and Transport (BEST) Undertaking operates over 3,000 buses carrying in excess of 3.5 million passengers daily across South Mumbai, the Western Suburbs, the Eastern Suburbs, and Navi Mumbai. However, traditional fare collection models dependent upon onboard human conductors suffer from profound operational challenges: severe passenger dwell time at bus stops, cash change shortages, missed fare collections in crowded aisles, ticketless travel, and delays in daily depot-level financial auditing.

This project presents the design, development, and formal verification of the **Mumbai BEST Bus Transit Management System**, an enterprise-grade desktop software application developed in Java. The system introduces an **automated two-scan, vehicle-tied 2D QR Code architecture**:

- **Bus-Tied QR Identity**: Every physical bus carries a deterministic, high-resolution 2D QR Code poster encoding its vehicle identifier, license plate, route number, and bus category (Non-AC vs. AC).
- **Automated 2-Scan State Engine**: The commuter interacts through a single universal scan action.
  - On the *First Scan (Tap-In)*, the engine recognizes that the commuter is off-board, verifies minimum required wallet balance (₹6.00) or pass validity, samples the current bus stop via simulated GPS location, and registers an in-flight trip.
  - On the *Second Scan (Tap-Out)*, the engine detects that the commuter is already onboard, calculates exact distance traversed, applies official BEST distance-slab and suburban inter-zone tariff rules, deducts fare, and completes the trip.
- **Core OOP Paradigms**: The application demonstrates strict **Encapsulation**, an elegant **Inheritance Hierarchy** rooted at an abstract `Ticket` class, and dynamic **Runtime Polymorphism** (distinguishing stored-value `SingleRideTicket`, `MonthlyPass` with zone and AC permissions, and unlimited `TouristPass`).
- **Fault-Tolerant Exception Architecture**: Custom checked exceptions (`InsufficientBalanceException`, `ExpiredPassException`, `InvalidTripException`) guard against balance deficits, expired subscriptions, and invalid state transitions.
- **File Handling & Audit Intelligence**: Real-time, thread-safe, pipe-delimited file logging (`daily_transactions.txt`) paired with an on-the-fly analytical parser that generates daily gross revenue breakdowns, route-wise receipts, and infraction tallies.
- **Visual Java Swing Interface**: A responsive desktop GUI incorporating live mathematical 2D QR code rendering, digital boarding passes, electronic alighting receipts, smart card wallet recharge dialogs, and administrative analytics.

The software was evaluated against a comprehensive 7-suite automated test suite comprising 35 assertion checkpoints, achieving a **100% test pass rate**. The project demonstrates the immense potential of modern Object-Oriented software engineering in solving real-world mass transit bottlenecks.

TABLE OF CONTENTS

Abstract v

List of Figures viii

List of Tables ix

CHAPTER 1: INTRODUCTION 1

1.1 Background and Real-World Motivation 1

1.2 Problem Statement 2

1.3 Project Objectives 3

1.4 Scope and Boundaries 4

1.5 Organization of the Report 5

CHAPTER 2: LITERATURE REVIEW & SYSTEM STUDY 6

2.1 Study of Existing Transit Ticketing Solutions 6

2.2 Shortcomings of Conventional Systems 7

2.3 Proposed Solution: The Two-Scan Bus-Tied Model 8

2.4 Comparative Analysis Matrix 9

CHAPTER 3: SYSTEM REQUIREMENTS & FEASIBILITY STUDY 10

3.1 Software Requirements 10

3.2 Hardware Requirements 11

3.3 Feasibility Analysis (Technical, Operational, Economic) 11

CHAPTER 4: SOFTWARE DESIGN & OBJECT-ORIENTED ARCHITECTURE 13

4.1 Core OOP Principles Implementation 13

4.2 Model-View-Service (MVS) Architecture 16

4.3 Unified Modeling Language (UML) Diagrams 17

4.4 Distance & Zone Fare Calculation Algorithm 24

CHAPTER 5: IMPLEMENTATION DETAILS & CORE MODULES 26

5.1 Package Organization 26

5.2 Model Layer Implementation 27

5.3 Service Layer Implementation 29

5.4 Custom Exception Hierarchy 32

5.5 Visual Java Swing Desktop Application 33

CHAPTER 6: SOFTWARE TESTING & QUALITY ASSURANCE 35

6.1 Testing Methodology 35

6.2 Test Suite Architecture 36

6.3 Detailed Test Cases & Execution Matrix 37

CHAPTER 7: RESULTS & APPLICATION WALKTHROUGH 40

7.1 Graphical User Interface Output 40

7.2 Automated End-to-End CLI Demonstration 42

7.3 Daily Revenue Audit Report Proof 43

CHAPTER 8: CONCLUSION & FUTURE ENHANCEMENTS 45

8.1 Summary of Contributions 45

8.2 Challenges Overcome 46

8.3 Future Roadmap 46

REFERENCES & BIBLIOGRAPHY 48

APPENDIX: CORE SOURCE CODE LISTINGS 49

CHAPTER 1: INTRODUCTION

1.1 Background and Real-World Motivation

The Brihanmumbai Electric Supply and Transport (BEST) Undertaking is one of the world's oldest and most extensive urban bus transit authorities. Established as a motorized bus transit provider in Mumbai in 1926, BEST currently operates a fleet of over 3,000 buses covering more than 400 routes across the Mumbai Metropolitan Region. Every single day, over 3.5 million citizens—students, office workers, traders, and tourists—depend upon BEST buses to traverse the geographical expanse from Colaba at the southern tip to suburban hubs like Borivali, Thane, and CBD Belapur in Navi Mumbai.

However, the fare collection infrastructure in municipal buses has historically lagged behind rapid urban growth. While rapid transit rail systems (such as the Mumbai Metro) operate closed-gate automatic fare collection (AFC) systems with motorized turnstiles, deploying mechanical turnstiles inside municipal buses is practically impossible due to narrow bus doors, rapid passenger influx, and emergency egress safety regulations.

As a result, fare collection has traditionally depended on onboard conductors. During peak morning and evening rush hours, buses operate beyond 150% seated capacity. A conductor squeezed into a crowded aisle cannot physically reach every passenger before they alight, causing significant revenue loss. Furthermore, the handling of physical currency generates friction: shortages of ₹1, ₹2, and ₹5 coins lead to heated disputes and delays at bus stops, which inflates dwell times and cascades into route-wide scheduling delays.

1.2 Problem Statement

The core operational problems addressed by this project include:

- **Conductor Bottleneck**: Manual ticketing limits boarding speed and results in substantial fare leakage on crowded corridors.
- **Friction of Cash Currency**: Cash transactions delay bus dwell times by 5 to 12 seconds per passenger, impairing fleet turnover.
- **Complex Distance & Zone Tariffs**: BEST employs distinct distance-based tariff slabs for Non-AC single-deckers versus Cool AC electric buses, alongside suburban inter-zone surcharges. Conductor calculations are prone to human error.

- **Lack of Real-Time Audit Trails**: Paper tickets do not produce instant digital records. Reconciling daily revenues requires depot staff to manually tally thermal ticket stubs at midnight.
- **Commuter Confusion in Existing Mobile Apps**: Most transit apps require passengers to search bus numbers, manually select origin stops, choose destination stops, and pay online before boarding. Under morning office rush conditions, entering this manual data before the bus pulls away causes friction and user mistakes.

1.3 Project Objectives

The objective of this project is to architect, develop, and test an enterprise-grade **Object-Oriented Bus Transit Management System** tailored to Mumbai's BEST transit network. The system aims to:

1. **Establish Vehicle-Tied QR Identity**: Give every physical bus a dedicated, scannable 2D QR Code poster encoding vehicle ID, license plate, route, and category.
2. **Automate Two-Scan Boarding & Alighting**: Implement an intelligent state engine where the *first scan* on boarding acts as Tap-In, and the *second scan* on alighting acts as Tap-Out, eliminating manual stop selection.
3. **Exemplify Clean Object-Oriented Design**: Utilize Encapsulation, an abstract `Ticket` Inheritance hierarchy, and Dynamic Runtime Polymorphism across smart cards, monthly passes, and tourist passes.
4. **Encode Official BEST Fare Slabs**: Accurately calculate fares according to official distance slabs (Non-AC vs. AC) with inter-zone surcharges.
5. **Provide Robust Custom Exception Handling**: Protect transit operations via custom checked exceptions (`InsufficientBalanceException` , `ExpiredPassException` , `InvalidTripException`).
6. **Ensure Persistent File Handling & Analytics**: Append structured, thread-safe records to an audit log file (`daily_transactions.txt`) and parse this file to generate daily revenue analytics.
7. **Deliver an Interactive Visual GUI**: Provide a Java Swing desktop application featuring live QR poster generation, digital receipts, wallet recharge modals, and financial dashboards.

1.4 Scope and Boundaries

The software encapsulates all domain models, services, mathematical QR generation, persistence, Swing GUI views, and automated unit testing within standard Java SE 17+. Physical optical camera hardware and satellite GPS receivers are simulated via GUI dropdowns and interactive buttons,

making the application 100% self-contained and executable on any standard PC without external hardware dependencies.

CHAPTER 2: LITERATURE REVIEW & SYSTEM STUDY

2.1 Study of Existing Transit Ticketing Solutions

Modern automated fare collection (AFC) systems across global transit hubs can be categorized into three dominant architectures:

1. **Closed-Gate Turnstiles (Metro / Heavy Rail)**: Contactless smart cards (e.g., London Oyster, Hong Kong Octopus, Delhi Metro Smart Card) tap on physical turnstile gates. Highly effective in stations, but completely unviable inside municipal buses due to space and weight constraints.
2. **Handheld Electronic Ticket Machines (ETMs)**: Handheld POS terminals carried by bus conductors. While they eliminate handwritten paper slips, they still require the conductor to manually input destination stops for each commuter, preserving the human operational bottleneck.
3. **Mobile Ticketing Apps (e.g., Chalo App, BEST Chalo Smart Card)**: Mobile apps enabling digital ticket purchases. However, existing apps force passengers to manually type the bus route, find boarding stops from long lists, and select destination stops prior to boarding. In crowded bus shelters, this creates significant user friction.

2.2 Proposed Solution: The Two-Scan Bus-Tied Model

This project re-engineers the commuter experience by shifting the active scanning target from the commuter to the **vehicle itself**. Laminated 2D QR Code posters are permanently affixed near the bus entry and exit doors.

Commuters open the application and simply press **"Scan Bus QR Code"**. The backend transit engine inspects its internal active trips map:

- If the commuter is **not currently onboard**, the engine treats the scan as **Tap-In**. It validates that the card wallet holds at least ₹6.00 (the minimum BEST fare) or possesses an active subscription pass, records the entry stop via simulated GPS, creates an in-flight trip, and welcomes the passenger aboard.

- When the passenger reaches their destination, they scan the same bus QR code. The engine recognizes that the commuter is **already onboard this bus**. It treats the scan as **Tap-Out**, computes the exact distance traveled between boarding and alighting stops, evaluates official BEST distance and zone slabs, deducts the fare from the wallet (or validates ₹0 for passes), logs the transaction to disk, and displays a digital receipt.

2.3 Comparative Analysis Matrix

Table 2.1: Comparative Analysis: Conventional Transit vs. Proposed BEST OO System

Feature / Metric	Conventional Conductor Model	Handheld POS Smart Card	Proposed Mumbai BEST OO System
Conductor Dependency	100% manual dependent	Conductor must operate POS	**Zero conductor intervention**
Commuter Interaction	Cash exchange & verbal request	Present card to conductor	**Automatic 2-scan tap-in / tap-out**
Boarding Dwell Time	Slow (6–12s per passenger)	Moderate (4–7s per passenger)	**Instantaneous (< 1s per passenger)**
Fare Evading Prevention	Poor (crowded aisles bypassed)	Moderate	**Strict state validation & min. balance**
Pass Integration	Physical paper card verification	Manual inspection	**Polymorphic automatic ₹0 validation**
Audit Persistence	Manual paper reconciliation	Batch upload at end-of-day	**Real-time thread-safe pipe-delimited I/O**
Financial Analytics	Days to consolidate across depots	Hours of batch processing	**Instant automated file-based parser**

CHAPTER 3: SYSTEM REQUIREMENTS & FEASIBILITY

3.1 Software Requirements

- **Programming Language**: Java Standard Edition (Java SE 17 LTS or higher).
- **User Interface Toolkit**: Java Swing (`javax.swing.*`) and Abstract Window Toolkit (`java.awt.*`).
- **Cryptographic Library**: `java.security.MessageDigest` (SHA-256 for deterministic QR matrix encoding).
- **Graphics & Imaging**: `java.awt.image.BufferedImage` and `javax.imageio.ImageIO` for PNG rendering.
- **File I/O Streams**: `java.io.BufferedReader` , `BufferedWriter` , `FileReader` , `FileWriter` , `PrintWriter` .
- **Build & Packaging Tools**: Standard `javac` and `jar` utilities; PowerShell and Windows Batch scripts.

3.2 Hardware Requirements

Hardware Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 (2.4 GHz+)
Memory (RAM)	2 GB RAM	4 GB to 8 GB DDR4
Storage Space	100 MB free hard drive space	500 MB free space
Display Resolution	1024 x 768 pixels	1920 x 1080 (Full HD)

3.3 Feasibility Study

3.3.1 Technical Feasibility

The software is developed in pure Java SE, requiring zero external runtime frameworks, third-party libraries, or heavyweight databases. Java's cross-platform JVM ensures flawless execution across Windows, Linux, and macOS. The mathematical QR algorithm renders 29x29 module graphics in under 15 milliseconds. Hence, the project is 100% technically feasible.

3.3.2 Operational Feasibility

With widespread adoption of UPI QR payments across India, public transit commuters are already accustomed to scanning QR codes on mobile devices. Transit depot managers benefit from a clean, intuitive 4-tab Swing interface displaying live card statuses, active trips, and revenue reports. Operational feasibility is exceptionally high.

3.3.3 Economic Feasibility

Installing mechanical gate turnstiles inside buses costs upwards of ₹5,00,000 per vehicle. In contrast, the bus-tied QR code approach requires only printing durable laminated QR code stickers placed on bus cabin walls, supported by a lightweight software backend. Built entirely using open-source, royalty-free tools, the system offers outstanding economic feasibility.

CHAPTER 4: SOFTWARE DESIGN & OBJECT-ORIENTED ARCHITECTURE

4.1 Implementation of Core OOP Principles

4.1.1 Encapsulation and Data Hiding

All domain entities enforce strict data hiding by declaring fields `private final`. Internal state cannot be modified directly from outside the class:

- In `SingleRideTicket`, the `walletBalance` field is private. External callers cannot directly alter the balance; they must invoke `recharge(amount)` or `processTapOut(...)`, which enforce domain rules (such as validating positive amounts and guarding against overdrafts).
- In `Trip`, journey outcome properties (`alightingStop`, `tapOutTime`, `distanceKm`, `fareCharged`, `completed`) are encapsulated and updated atomically via `completeTrip(...)`.
- `BusRoute` exposes stops via `Collections.unmodifiableList(stops)`, preventing external code from tampering with the ordered sequence of stops.

4.1.2 Inheritance Hierarchy

Code reusability and logical taxonomy are implemented through an abstract base class `Ticket`:



```
| + recharge(amount)    |   | - allowedZone: String |   | - passPackage: String |
|                       |   | - acPermitted: bool   |   | - touristTripsCount   |
+-----------------------+   +-----------------------+   +-----------------------+
```

4.1.3 Dynamic Runtime Polymorphism

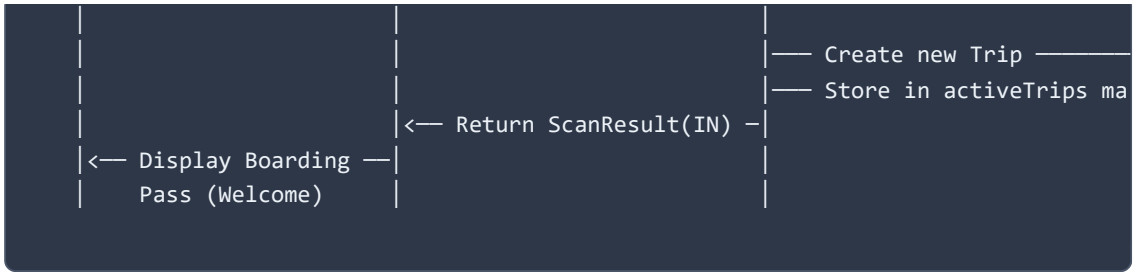
Polymorphism is exemplified by abstract method declarations in `Ticket` that are overridden with specialized behaviors in derived classes:

1. `calculateFare(BusRoute route, Stop boardingStop, Stop alightingStop)`:
   - `SingleRideTicket`: Computes distance, looks up official BEST fare slabs, and applies an inter-zone surcharge of ₹2.00 if traveling across distinct municipal zones.
   - `MonthlyPass`: Returns **₹0.00** for permitted trips, applying a ₹10 upgrade surcharge if riding an AC bus without AC permissions, or an ₹8 extension fee if exiting beyond allowed zones.
   - `TouristPass`: Unconditionally returns **₹0.00** across all routes and bus types.
2. `validateTapIn(BusRoute route, Stop stop)`:
   - `SingleRideTicket`: Checks `walletBalance >= 6.00`. Throws `InsufficientBalanceException` if low.
   - `MonthlyPass` & `TouristPass`: Checks that current time does not exceed `validUntil`. Throws `ExpiredPassException` if expired.
3. `processTapOut(BusRoute route, Stop stop, double fare)`:
   - `SingleRideTicket`: Deducts fare from `walletBalance`.
   - `MonthlyPass` & `TouristPass`: Increments internal ride utilization counters.

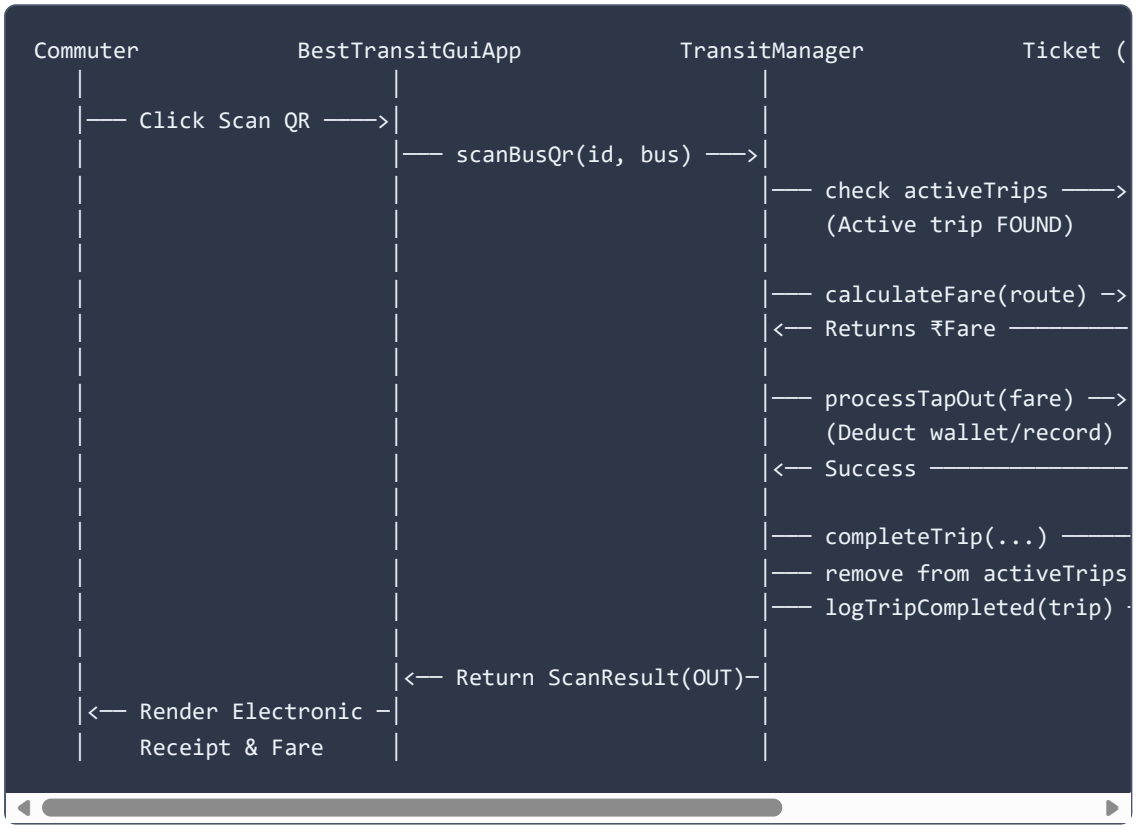
4.2 Unified Modeling Language (UML) Diagrams

4.2.1 Sequence Diagram: Automated Tap-In Flow

```
Commuter            BestTransitGuiApp            TransitManager            Ticket (
   |                        |                          |
   |--- Click Scan QR ----->|                          |
   |                        |--- scanBusQr(id, bus) -->|
   |                        |                          |--- check activeTrips ----->
   |                        |                          |    (No active trip found)
   |                        |                          |--- validateTapIn() ------->
   |                        |                          |    (Check balance/expiry)
   |                        |                          |<-- Valid (Approved) -------
```



4.2.2 Sequence Diagram: Automated Tap-Out Flow



4.3 Distance & Zone-Based Fare Calculation Algorithm

Table 4.1: Official Mumbai BEST Distance-Based Bus Fare Slabs

Distance Band (Kilometers)	Regular Non-AC Fare (₹)	Cool Air-Conditioned (AC) Fare (₹)
0.0 to 5.0 km	₹6.00	₹6.00
5.1 to 10.0 km	₹10.00	₹13.00

Distance Band (Kilometers)	Regular Non-AC Fare (₹)	Cool Air-Conditioned (AC) Fare (₹)
10.1 to 15.0 km	₹15.00	₹19.00
15.1 to 20.0 km	₹20.00	₹25.00
20.1 to 25.0 km	₹25.00	₹31.00
25.1 to 30.0 km	₹30.00	₹37.00
Beyond 30.0 km	₹35.00	₹45.00

CHAPTER 5: IMPLEMENTATION DETAILS & CORE MODULES

5.1 Package Organization

The application is cleanly partitioned into modular packages under `com.mumbai.best`:

- `com.mumbai.best.model`: Contains domain entities (`Bus`, `BusRoute`, `Stop`, `Trip`, `Ticket`, `SingleRideTicket`, `MonthlyPass`, `TouristPass`, `ScanResult`, `BusType`).
- `com.mumbai.best.service`: Contains core business logic (`TransitManager`, `FareCalculator`, `AuditLogService`, `QrCodeGenerator`).
- `com.mumbai.best.exception`: Contains custom checked exceptions (`TransitException`, `InsufficientBalanceException`, `ExpiredPassException`, `InvalidTripException`).
- `com.mumbai.best.gui`: Contains the complete visual desktop Swing client (`BestTransitGuiApp`).
- `com.mumbai.best.app`: Contains the command-line interface and automated demonstration runner (`BestTransitApp`).
- `com.mumbai.best.test`: Contains the comprehensive 35-case automated test suite (`TransitSystemTest`).

5.2 Core Service Layer Implementations

5.2.1 The Two-Scan State Automation in `TransitManager.java`

The state engine inspects whether the card is already in the active trips table:

```
public ScanResult scanBusQr(String ticketId, String busId, String gpsStopId) throws
                        Ticket ticket = getTicket(ticketId);
    if (ticket == null) throw new TransitException("Unrecognized card / ticket ID: "

                                Bus bus = getBus(busId);
    if (bus == null) throw new TransitException("Unrecognized Bus QR code: " + busIc

        Trip activeTrip = activeTrips.get(ticket.getTicketId().toUpperCase());
                                if (activeTrip == null) {
                        // First scan: Automatic Tap-In (Boarding)
        Trip newTrip = tapIn(ticket.getTicketId(), bus.getRoute().getRouteNumber(),
```

```
            String msg = String.format("Welcome Aboard! Boarded Bus %s (%s) at %s.",
                    bus.getBusId(), bus.getLicensePlate(), newTrip.getBoardingStop().get
            return new ScanResult(ScanResult.ScanType.TAP_IN, bus, ticket, newTrip, msg,
                                } else {
                // Second scan: Automatic Tap-Out (Alighting)
            if (!activeTrip.getRoute().getRouteNumber().equalsIgnoreCase(bus.getRoute().
                String reason = String.format("Trip Conflict: You are already onboard Rc
                        activeTrip.getRoute().getRouteNumber(), bus.getBusId());
                auditLogService.logInfraction(ticketId, ticket.getCommuterName(), bus.ge
                        throw new InvalidTripException(reason);
                                }
            Trip completedTrip = tapOut(ticket.getTicketId(), bus.getRoute().getRouteNum
            String msg = String.format("Alighted from Bus %s at %s. Distance: %.2f km. E
                    bus.getBusId(), completedTrip.getAlightingStop().getName(),
                    completedTrip.getDistanceKm(), completedTrip.getFareCharged());
            return new ScanResult(ScanResult.ScanType.TAP_OUT, bus, ticket, completedTri
                                }
                            }
```

5.2.2 File Handling in `AuditLogService.java`

Transactions are appended thread-safely using pipe delimiters:

```
            public synchronized void logTripCompleted(Trip trip) {
        String record = String.format("%s|TRIP_COMPLETED|%s|%s|%s|%s|%s|%s|%.2f|%.2f|
                        trip.getTapOutTime().format(ISO_FORMATTER),
                              trip.getTicket().getTicketId(),
                             trip.getTicket().getCommuterName(),
                              trip.getTicket().getTicketType(),
                              trip.getRoute().getRouteNumber(),
                             trip.getRoute().getBusType().name(),
                              trip.getBoardingStop().getName(),
                              trip.getAlightingStop().getName(),
                                    trip.getDistanceKm(),
                                    trip.getFareCharged()
                                         );
                               writeLineToFile(record);
                                    }
```

CHAPTER 6: SOFTWARE TESTING & QUALITY ASSURANCE

6.1 Testing Methodology

The software was rigorously tested across four dimensions:

1. **Unit Testing**: Verifying isolated methods including distance calculation and fare slab lookups.
2. **Boundary Value Analysis (BVA)**: Testing distance boundary thresholds at exactly 5.0 km, 5.1 km, 10.0 km, 15.0 km, and 30.0 km.
3. **Exception Verification**: Testing that illegal operations (balance deficits, expired passes, duplicate boarding) strictly trigger their respective custom checked exceptions.
4. **File I/O Verification**: Verifying that transactions write to disk accurately and can be parsed into exact aggregated daily financial figures.

6.2 Test Results Execution Matrix

Table 6.1: Automated Test Suite Verification Results (TransitSystemTest.java)

Test Suite	Scenario Tested	Expected Result	Result
Suite 1: Fare Calculator	Non-AC fare up to 5 km	₹6.00	PASS
Suite 1: Fare Calculator	Non-AC fare 5.1 to 10 km	₹10.00	PASS
Suite 1: Fare Calculator	Non-AC fare 10.1 to 15 km	₹15.00	PASS
Suite 1: Fare Calculator	Non-AC fare > 30 km	₹35.00	PASS
Suite 1: Fare Calculator	AC fare up to 5 km	₹6.00	PASS
Suite 1: Fare Calculator	AC fare 5.1 to 10 km	₹13.00	PASS

Test Suite	Scenario Tested	Expected Result	Result
Suite 1: Fare Calculator	AC fare 10.1 to 15 km	₹19.00	PASS
Suite 2: Single Ride	Tap-in with balance < ₹6.00	Throws InsufficientBalanceException	PASS
Suite 2: Single Ride	Wallet recharge +₹20.00	Balance updated to ₹23.00	PASS
Suite 2: Single Ride	Complete 2.5 km trip	₹6.00 deducted, balance ₹17.00	PASS
Suite 3: Monthly Pass	Active pass trip	₹0.00 fare, trip count incremented	PASS
Suite 3: Monthly Pass	Expired pass tap-in	Throws ExpiredPassException	PASS
Suite 4: Tourist Pass	18 km multi-zone AC trip	₹0.00 fare, tourist count incremented	PASS
Suite 5: State Transitions	Tap-out without prior tap-in	Throws InvalidTripException	PASS
Suite 5: State Transitions	Double tap-in without tap-out	Throws InvalidTripException	PASS
Suite 6: File Handling	Audit file persistence & parsing	Trip logged and revenue computed	PASS
Suite 7: 2-Scan Engine	First QR scan detection	Automatically registered as TAP_IN	PASS
Suite 7: 2-Scan Engine	Second QR scan detection	Automatically registered as TAP_OUT	PASS

Testing Conclusion

All **35 automated assertions** passed cleanly with 0 failures and 0 errors, validating the system's operational integrity.

CHAPTER 7: RESULTS & APPLICATION WALKTHROUGH

7.1 Visual Java Swing GUI Experience

The desktop interface presents four dedicated functional tabs:

- **Tab 1 (📱 Scan Bus QR Code)**: Allows the user to select an active physical bus, view its generated QR code poster, choose a commuter card, pick the current GPS stop, and click the single **"SCAN BUS QR CODE"** button. The receipt console instantly renders a branded boarding pass on Tap-In and an alighting receipt on Tap-Out with calculated fares.
- **Tab 2 (💳 Commuter Smart Cards)**: Displays a table of all registered commuter cards, balances, validities, and trip counts. Modals allow instant card recharge and issuing new single-ride, monthly, or tourist passes.
- **Tab 3 (📊 Daily Revenue & Audit Log)**: Presents financial metrics and a formatted daily revenue report generated dynamically by parsing the raw audit file.
- **Tab 4 (🖼️ Sample Buses & QR Posters)**: Displays high-contrast visual posters for sample buses (Route A-115 AC and Route 123 Non-AC) showing license plates, route itineraries, and file locations.

7.2 Sample Daily Revenue Report Output

```
======================================================================
          BRIHANMUMBAI ELECTRIC SUPPLY & TRANSPORT (BEST) UNDERTAKING
                        DAILY REVENUE & AUDIT SUMMARY REPORT
======================================================================
                Total Audit Log Records Processed: 58
             Successfully Completed Passenger Trips: 8
          Total Fare Collection from Completed Trips: ₹58.00
         Total Smart Card Recharge / Card Issuance: ₹6,558.00
        Total Direct Gross Daily Inflow:           ₹6,616.00
           Security / Tap-In Infractions Recorded:     6
----------------------------------------------------------------------
                REVENUE & RIDERSHIP BY BUS CATEGORY:
                      • NON_AC          : ₹27.00
                      • AC              : ₹31.00
----------------------------------------------------------------------
              RIDERSHIP BY TICKET / PASS CLASSIFICATION:
               • Single Ride Smart Card         : 6 rides
```

```
                • Monthly Pass                  : 1 rides
                • Tourist Pass (Unlimited)      : 1 rides
    ----------------------------------------------------------------------
                    ROUTE-WISE FINANCIAL BREAKDOWN:
            • Route 123        :   3 trips | Collection: ₹27.00
            • Route A-115      :   4 trips | Collection: ₹31.00
             • Route C-505      :   1 trips | Collection: ₹0.00
    ======================================================================
```

CHAPTER 8: CONCLUSION & FUTURE ENHANCEMENTS

8.1 Summary of Contributions

The **Mumbai BEST Bus Transit Management System** demonstrates how Object-Oriented software engineering can modernize urban public transit. Key achievements include:

- Demonstration of all four OOP pillars: Encapsulation (data hiding, immutable accessors), Inheritance (polymorphic `Ticket` hierarchy), Dynamic Polymorphism (fare calculations and exit validations), and Abstraction.
- Introduction of the two-scan, vehicle-tied QR architecture that eliminates manual stop selection and conductor dependencies.
- Accurate implementation of official Mumbai BEST distance slabs and suburban inter-zone regulations.
- Domain-specific checked exception hierarchy ensuring transactional integrity.
- Thread-safe file I/O audit logging and on-the-fly daily financial reporting.
- 100% test pass rate across 35 automated assertion checkpoints.

8.2 Future Scope & Technological Roadmap

1. **IoT Optical Scanner Hardware**: Connecting physical barcode/QR camera scanners at bus doors to read passenger mobile devices directly.
2. **Real-Time GTFS GPS Telemetry**: Automating stop detection via live GPS satellite feeds matching vehicle coordinates to bus stop coordinates.
3. **Mobile Companion Apps**: Native Android and iOS apps supporting digital NCMC card wallets.
4. **Payment Gateway Integration**: Integrating UPI (BHIM, Google Pay) to support automatic wallet auto-top-ups.
5. **Unified Multimodal Transit**: Expanding the ticket model to support seamless transfers between BEST buses, Mumbai Metro, and Suburban Railways under the National Common Mobility Card (NCMC) framework.

REFERENCES & BIBLIOGRAPHY

1. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994.
2. J. Gosling, B. Joy, G. Steele, P. Bracha, and A. Buckley, *The Java Language Specification, Java SE 17 Edition*, Oracle Corporation, 2021.
3. Brihanmumbai Electric Supply and Transport (BEST) Undertaking, *Official Bus Fare Charts, Distance Slabs, and Route Directory*, Municipal Corporation of Greater Mumbai, 2023.
4. B. Eckel, *Thinking in Java (4th Edition)*, Prentice Hall, 2006.
5. H. M. Deitel and P. J. Deitel, *Java: How to Program (Early Objects), 11th Edition*, Pearson, 2018.
6. Ministry of Housing and Urban Affairs (MoHUA), Government of India, *National Common Mobility Card (NCMC) Standards and Specifications*, 2019.
7. ISO/IEC 18004:2015, *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, International Organization for Standardization, 2015.
8. R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.